Lean 4 Cheat Sheet

PoPL TAs

August 14, 2025

1 Basics

Hello World

Entry point for programs; similar to `main()` in other languages

```
def main : IO Unit :=
  IO.println "Hello, Lean!"
```

Comments

```
-- Single line comment
/- Multi
  line comments -/
```

Variables

Immutable by default; use `let mut` for mutability.

```
def x := 42          -- immutable by default
def y : String := "Hi"
```

Local bindings, similar to `let` in JS or variable assignment in Python.

```
def z :=
  let a := 10;
  a + 5
def foo :=
  let a := 5
  let a := a + 1
  a
def demo : IO Unit := do
  let mut a := 5
  a := a + 1
  IO.println a
```

Functions

```
def add (a b : Int) : Int := a + b
def greet name :=
  "Hello, " ++ name
```

Type Annotations

```
def combine (x : Int) (y : String) : String :=
  s!"{x} and {y}"
```

Abbreviations

`abbrev` creates a simple alias (no new type).

```
abbrev Name := String
abbrev A := Int → Int
abbrev B := func <| a
```

2 Data Types

Basic Types

```
Nat,-- natural numbers (0, 1, 2, ...), non-negative and default.
-- `Int` is for integers (can be negative) and are arbitrary-precision.
Int, Int8, Int16, Int32, Int64, UInt, UInt8, UInt16, UInt32, UInt64,
ISize, USize, Float, Float32, String, Bool, Char,
List Int, Array String, Vector Float 5,
Option Int  -- Some 5 or None; ... and more
```

Type hierarchy & Unit type

- Types themselves have types: `Type : Type 1`

```
def foo : Type := Nat
```

Used when a function returns "nothing" (like `void` in JS or `None` in Python).

```
def doNothing : Unit := ()
```

Type inference

```
def n := 5        -- type inferred as Int
def s := "lean"   -- String
```

Booleans & Logic

```
def t := true && false
def u := if t then 1 else 0
```

Sum and Product Types

```
-- Sum types: enums and tagged unions
inductive Color
  | red
  | green
  | blue
def fav : Color := .green
-- Tagged unions with data
inductive Shape
  | circle (radius : Float)
  | rectangle (w : Float) (h : Float)
  | point
def s1 : Shape := .circle 2.5
def s2 : Shape := .rectangle 3.0 4.0
-- Pattern matching on sum types
def area : Shape → Float
  | .circle r      => 3.1415 * r * r
  | .rectangle w h => w * h
  | .point         => 0.0

-- Product types: tuples and custom structures
def tup : (Int × String) := (42, "hello")
structure Person where
  name : String
  age : Nat
def alice : Person := { name := "Alice", age := 30 }
def alice' : Person := ( "Alice", 30 )
def alice'' : Person := { alice with age := 31 }
alice.name
```

3 Lists & Arrays

Lists

```
def nums := [1,2,3]                -- List Int
def more := nums ++ [4,5,6]        -- append
def add := 45 :: more              -- prepend
nums[2]                            -- indexing
nums[4]!                           -- unsafe indexing (may panic)
nums[4]?                           -- safe indexing, returns Option
nums.length                        -- length
List.range 5                       -- [0,1,2,3,4]
nums.foldl (fun acc x => acc + x) 0   -- left fold (reduce)
nums.foldr (fun x acc => acc - x) 0   -- right fold
nums.map (fun x => x + 1)
nums.filter (fun x => x % 2 == 0)
```

Arrays

```
def arr := #[1, 2, 3]   -- Array
arr.push 4
arr.pop
arr.push 4              -- append (returns new array)
arr[2]                  -- indexing
arr[4]?                 -- safe indexing
arr[1:3]                -- slice (no skip like Python)
arr[:3]
arr[1:]
Array.replicate 5 0     -- create array of length 5, filled with 0
def newArr := arr.set 1 99   -- returns a new array with index 1 = 99
```

Vectors

Vector is an Array with fixed size

```
def v : Vector Int 3 := (#[1, 2, 3], rfl)
```

Tuples

```
def pair := (1, "a")
pair.1                  -- 1
pair.2                  -- "a"
pair.fst
pair.snd
def triple := (1, "a", 3)
triple.1                -- 1
triple.2                -- ("a", 3)
```

Range

Similar to Python's `range`, you can create ranges in Lean.

```
[:5]
[4:5]
def a := [1:10:2]  -- start, end, step
#eval a.size
-- [5:0:-1]  -- NOT Possible as `-1` is not `Nat`
```

4 Pattern Matching

On values

```
def describe : Int → String
| 0 => "zero"
| 1 => "one"
| _ => "many"
```

On tuples

```
def swap : (Int × Int) → (Int × Int)
| (a, b) => (b, a)
```

Match expressions in let bindings:

```
def exprMatch :=
  let (a, b) := (1, 2)
  a + b
```

On lists

```
def sumList : List Int → Int
| []      => 0
| x :: xs => x + sumList xs
```

Wildcard usage in destructuring:

```
def onlyHead : List Int → Option Int
| x :: _ => some x
| []     => none
```

On options

```
def bar : Option Nat → Nat
  | some n => n + 1
  | none   => 0
```

5 Loops & Recursion

For in range

```
def loopRange : IO Unit := do
  for i in [0:5] do
    IO.println s!"i = {i}"
def loopList (xs : List Int) : IO Unit := do
  for x in xs do
    IO.println x
def loopArray (arr : Array Int) : IO Unit := do
  for x in arr do
    IO.println x
```

While loop

```
def loop' (n : Int) : IO Unit := do
  let mut i := n
  while i > 0 do
    IO.println i
    i := i - 1
```

Recursion and `let rec`

```
def fact : Nat → Nat
| 0 => 1
| n+1 => (n+1) * fact n
def factTail (n : Nat) : Nat :=
  let rec go (n acc : Nat) :=
    if n <= 0 then
      acc
    else
      go (n - 1) (acc * n)
  go n 1
```

Mutual Recursion

```
mutual
  def even : Nat → Bool
  | 0 => true
  | n+1 => odd n
  def odd : Nat → Bool
  | 0 => false
  | n+1 => even n
end
```

Partial Definitions

Use `partial def` for recursion Lean can't prove terminates.

```
partial def fac (n : Nat) : Nat :=
  if n == 0 then 1 else n * fac (n - 1)
```

Functions

```
def add (x y : Int) : Int := x + y
def add2 := (fun x => x + 2)
def qsort (xs : List Int) : List Int :=
  let rec go (xs : List Int) (lt : List Int) (gt : List Int) :=
    match xs with
    | [] => lt ++ gt
    | x :: xs =>
      let pivot := if lt.isEmpty then 0 else lt.head!
      if x < pivot
        then go xs (lt ++ [x]) gt
        else go xs lt (gt ++ [x])
  go xs [] []

-- Higher-order functions
def nums := [1, 2, 3]
def doubled := nums.map fun x => x * 2
def evens := nums.filter fun x => x % 2 == 0

def foo {α : Type} (x : α) := x
-- `{}` means the argument is implicit and can be inferred.
#check foo 5

-- Function composition and pipeline
def f (x : Int) := x + 1
def g (x : Int) := x * 2
def h := f ∘ g                -- h(x) = f(g(x))
def result := 3 |> g |> f     -- result = f(g(3))
def result' := g <| f <| 3    -- result = g(f(3))
def result'' := g ∘ f         -- function composition g(f(_))
```

6 Strings

```
def c : Char := 'a'
def s := "LeaN"
s ++ " rocks"           -- concat
s.length                -- length
s.get 0                 -- Char (not String)
s.toUpper               -- uppercase
s.toLower               -- lowercase
s.capitalize            -- capitalize
s.drop 2                -- drop first 2 chars (by codepoint, not byte)
s.take 2                -- take first 2 chars
s.splitOn "a"           -- split by substring
s.trim                  -- remove whitespace
s.isEmpty               -- check if empty
", ".intercalate ["a", "b", "c"]        -- "a,b,c"
String.join ["a", "b", "c"]             -- "abc"
s.contains 'a'          -- check if contains substring
s.extract (1) (3)       -- Get substring
```

7 Modules & Imports

```
-- file: Math/Utils.lean
def square (x : Int) := x * x
private def secret := 42  -- this is not accessible outside this file
protected def hidden := 99  -- this is accessible in the same module

-- in another file
import Math.Utils         -- absolute import
import ..Math.Utils       -- relative import (from parent module)
def r := square 5

open Nat                  -- brings all of Nat's definitions into scope
open Nat (succ)           -- only bring `succ` into scope
open Nat hiding succ      -- bring all except `succ`
def n := succ 4           -- can use Nat.succ as succ
export Nat (succ, pred)   -- only export `succ` and `pred`
-- to Avoid name clashes
namespace MyMath
  open scoped Nat
  def triple (x : Int) := x * 3
end
def t := MyMath.triple 4

-- Local scoping
section
  variable {α : Type}
  def id (x : α) := x
end
```

8 Typeclasses & Instances

Typeclass basics

```
structure Point where
  x : Int
  y : Int
  deriving Repr, DecidableEq

#eval toString { x := 1, y := 2 }   -- needs Repr
```

Defining a class and instance

```
class AddOne (α : Type) where
  addOne : α → α

instance : AddOne Int where
  addOne x := x + 1

def inc (x : Int) := AddOne.addOne x
```

9 Prop vs Type

Prop vs Type

```
def isEven (n : Nat) : Prop := ∃ k, n = 2 * k
def double (n : Nat) : Nat := 2 * n
```

10 Error Handling

Except, ExceptT, OptionT

```
def safeDiv (x y : Int) : Except String Int :=
  if y == 0 then Except.error "divide by zero" else Except.ok (x / y)

def safeDivOpt (x y : Int) : Option Int :=
  if y == 0 then none else some (x / y)

-- map on Option
(some 2).map fun x => x + 1  -- => some 3
```

11 Do Notation for Monads

Do notation outside IO

```
def addOpt (a b : Option Int) : Option Int := do
  let x ← a
  let y ← b
  pure (x + y)

def addExc (a b : Except String Int) : Except String Int := do
  let x ← a
  let y ← b
  pure (x + y)
```

12 Interactive Mode

Interactive commands

```
#check 2 + 2         -- show type
#eval 2 + 2          -- evaluate
#reduce (2 + 2) * 3  -- reduce expression
```

13 IO & Side Effects

```
def main : IO Unit := do
  IO.println "Enter a number:"
  let input ← IO.getLine
  let n := input.toInt!    -- convert String to Int (unsafe, may throw)
  IO.println s!"You entered: {n}"

-- Mixing IO and pure code: keep pure functions separate from IO.
def double (x : Int) := x * 2   -- pure function

def main2 : IO Unit := do
  IO.println "Enter a number:"
  let input ← IO.getLine
  let n := input.toInt!
  let result := double n      -- use pure function in IO
  IO.println s!"Double: {result}"
```

14 Additional notes

- In dependent type theory, everything is a term, and every term has a type.
  - We write `t : T` to indicate that `t` is a term of type `T`.

15 Notes for Python/JS Users

- Lean is compiled, not interpreted — changes require recompilation.
- No implicit type conversions: Nat, Int, Float must be converted explicitly (e.g., `Int.ofNat`, `Float.ofInt`).
- `:=` in `let` binds a value immediately; it's not like JS destructuring or "assign later."
- Everything is **immutable** unless marked `mut`.
- `def` / `let` use `:=` to bind/define values.
- `=` is the **equality proposition** (`a = b : Prop`) — a logical statement.
- `==` is the Boolean equality operator (returns `Bool`).
- `IO` is like async/await — pure code and side-effect code are separate.
- No `null`; use `Option` for maybe-values, and `Unit` for no-value.
- No loops for `List` — use recursion or `for ... in`.

16 References

- Lean Home page